

ProfileDataCard.js — Fundamentos

Subcomponente reutilizable de tarjeta de datos de perfil (GymBro App)

Es un componente nuevo, creado por refactor a partir de un patrón que se repetía casi idéntico en `mi_cuenta_trainer.js` y `mi_cuenta_user.js`: dos “tarjetas” de datos de perfil (“Datos Personales”, “Contacto y Salud”), cada una con varias filas de ícono + etiqueta + valor, escritas a mano fila por fila.

1. Por qué se diseñó así

```
export default function ProfileDataCard({ title, rows }) {
  return (
    <View style={styles.card}>
      <Text style={styles.sectionTitle}>{title}</Text>

      {rows.map((row) => (
        <View key={row.label} style={styles.dataRow}>
          <Text style={styles.icon}>{row.icon}</Text>
          <View>
            <Text style={styles.label}>{row.label}</Text>
            <Text style={styles.value}>{row.value}</Text>
          </View>
        </View>
      ))}
    </View>
  );
}
```

Recibe **dos props**:

- **title**: el encabezado de la tarjeta (“Datos Personales”, “Contacto y Salud”).
- **rows**: un **array de objetos** { icon, label, value }, uno por cada fila de dato a mostrar.

En vez de escribir 5 (o 4) bloques `<View style={styles.dataRow}>...</View>` casi idénticos a mano, la pantalla ahora **describe los datos como datos** (un array), y el componente los recorre con `.map()` para generar las filas. Es exactamente el mismo patrón que ya se usa dos veces antes en el proyecto:

- `OptionChips` en `signup.js` (recorre `GENDERS/GOALS` para generar pastillas).
- El `.map()` sobre `usuariosFiltrados` en `buscar_usuario_trainer.js` (recorre usuarios para generar tarjetas).

`key={row.label}` cumple el mismo rol de siempre: como label es único dentro de cada tarjeta (“DNI”, “Género”, “Edad”...), sirve perfectamente como key sin necesitar un id numérico extra.

2. Por qué es reutilizable para ambas pantallas

El componente **no sabe nada** sobre si lo está usando un entrenador o un alumno — no importa `useAuth()`, no lee `user` directamente. Todo lo que necesita mostrar le llega por props. Esto significa que sirve exactamente igual para las dos tarjetas de `mi_cuenta_trainer.js` (ya migrado) y para las dos tarjetas equivalentes de `mi_cuenta_user.js` (que tiene la estructura idéntica, así que se podría migrar con el mismo criterio).

3. Estilos autocontenidos

El componente trae su **propio** StyleSheet (card, sectionTitle, dataRow, icon, label, value) — no depende de que la pantalla que lo usa tenga esos nombres de estilo definidos. Esto es justamente lo que permitió **eliminar** esos seis estilos de `mi_cuenta_trainer.js`: ya no hacen falta ahí, viven exclusivamente dentro de `ProfileDataCard.js`.

4. Cómo quedó `mi_cuenta_trainer.js`

```
<ProfileDataCard
  title="Datos Personales"
  rows={[
    { icon: '■', label: 'DNI', value: user.dni },
    { icon: '■', label: 'Género', value: user.gender },
    { icon: '■', label: 'Edad', value: user.age },
    { icon: '■', label: 'Peso', value: user.weight },
    { icon: '■', label: 'Altura', value: user.height },
  ]}
/>

<ProfileDataCard
  title="Contacto y Salud"
  rows={[
    { icon: '☎', label: 'Teléfono', value: user.phone },
    { icon: '☎', label: 'Tel. Emergencia', value: user.emergencyPhone },
    { icon: '■', label: 'Obra Social', value: user.healthInsurance },
    { icon: '■', label: 'Contraindicaciones', value: user.contraindications },
  ]}
/>
```

De ~110 líneas de JSX repetitivo pasó a 18 líneas declarativas. Si mañana quisieran cambiar el estilo de una fila (por ejemplo, el tamaño de fuente del value), se edita en **un solo lugar** (`ProfileDataCard.js`) y el cambio se refleja automáticamente en ambas tarjetas, en vez de tener que buscar y tocar 9 bloques repetidos.

5. Otros dos cambios aplicados junto con este refactor

- **isLandscape faltante:** `TrainerBottomNav` ahora recibe `isLandscape={isLandscape}` en `mi_cuenta_trainer.js`, así que en modo horizontal la barra inferior de esta pantalla ajusta correctamente su altura, igual que en el resto de las pantallas (antes quedaba con el alto de portrait sin importar la orientación).
- **Estilos huérfanos eliminados:** se quitaron de `mi_cuenta_trainer.js` los estilos `card`, `sectionTitle`, `dataRow`, `icon`, `label`, `value` (ahora viven en `ProfileDataCard.js`) y `objective/objectiveIcon/objectiveText` (restos de haber copiado `mi_cuenta_user.js`, sin ningún JSX que los usara en esta pantalla).

Resumen para el oral

“ProfileDataCard es un componente de presentación puro (sin hooks, sin lógica de negocio) que recibe un título y un array de filas {icon, label, value}, y las recorre con .map() para dibujar una tarjeta de datos de perfil. Nació de identificar que mi_cuenta_trainer.js y mi_cuenta_user.js repetían la misma estructura fila por fila, y se diseñó sin ninguna dependencia de a quién pertenece el perfil, así que sirve igual para el entrenador que para el alumno.”